

Nombre: Nahuel
Apellido: Toloza
Año y Curso: 5to 2da
Escuela: E.T 32

Propósito de esto: Este archivo será empleado como una "carpeta" para el Examen de P.O.O , por lo que dejare aqui todo lo que registre en taller con texto, imagenes y demas

¿Qué es P.O.O?

Programación Orientada a Objetos, P.O.O o simplemente taller (para mi), es una "materia" (en el contexto escolar) centralizado en la organización y registro de datos u "objetos" con el fin de representarlos de forma modular y reutilizable, ya sea en gráficos o código

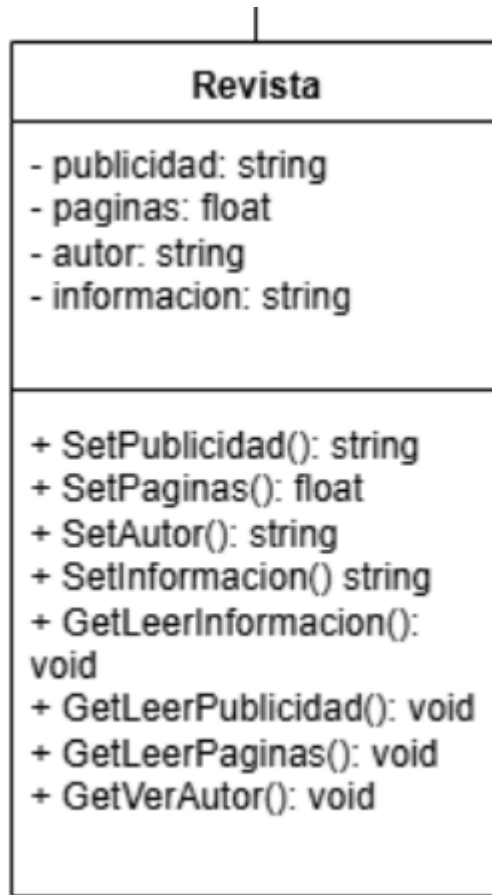
Definiciones Básicas

- Objeto: es una instancia de una clase. posee tanto atributos como métodos o datos y acciones respectivamente
- Clase: un modelo que cuyo propósito es definir la estructura y el comportamiento de los objetos al darle un propósito/uso/definición claro a los atributos como los métodos de los objetos
- Atributos: cualidades o características de los objetos
- Métodos: acciones o funciones de los objetos que se pueden realizar o ejecutar

Pilares de P.O.O

- Herencia: hace que las clases hijas, o subclases, "hereden" (de ahí su nombre) tanto los atributos o métodos de una clase, esto permite a las subclases usar las funciones de la superclase al reutilizar código/texto o alterar y modificar su propio comportamiento. establece una especie de "jerarquía" donde la clase madre (por así decirlo) está encima de las clases hijas que dependen de dicha clase madre
- Polimorfismo: a diferencia de Herencia, el polimorfismo hace que los métodos de la clase madre sean "reciclados" pero con el cambio notable de ajustarlos de forma específica. es decir: mientras que herencia reutiliza, polimorfismo los modifica para algo en específico pero respondiendo al mismo mensaje
- Encapsulamiento: Su propósito es el de clasificar y asegurar los datos ingresados o por ingresar, siendo el símbolo + para públicos, - menos para privados y # para indicar que solo las clases hijas o subclases pueden acceder a esos datos
- Abstracción: su función es la reducción de la complejidad del sistema al ocultar detalles innecesarios y centrarse en lo importante, simplificando los objetos

Cosos de importancia

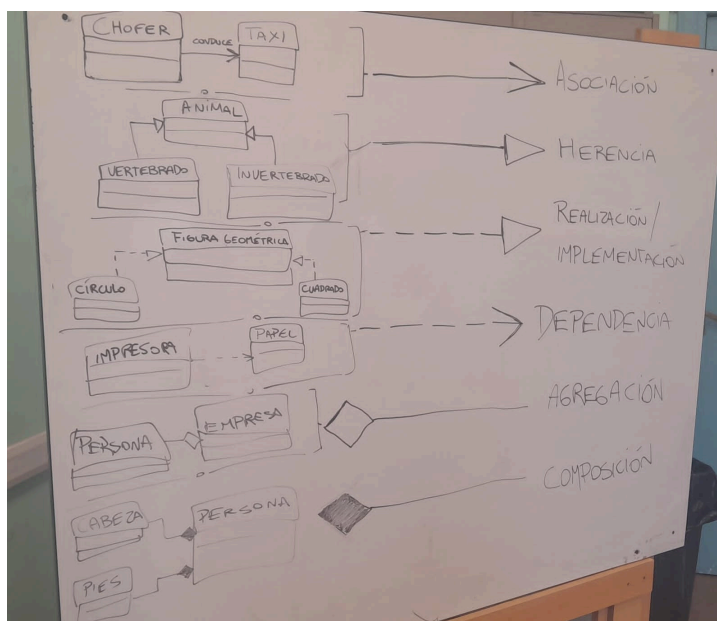


Set: se utiliza cuando se quiere establecer los datos

Get: cuando se quiere obtener datos

- = privado
- + = público
- # = protegido

Flechas



- Asociación: Flecha de cuerpo normal pero punta hueca sin la "base"
- Herencia: Flecha de cuerpo normal pero de punta hueca CON la "base"
- Implementación: Flecha de cuerpo entrecortado de punta hueca CON la "base"
- Dependencia: Flecha de cuerpo entrecortado pero con la punta hueca sin la "base"
- Agregación: Flecha carente de punta pero con un rombo hueco en la zona de donde emerge
- Composición: Flecha carente de hueca pero con un rombo negro en la zona de donde emerge

0:0 ninguno a ninguno

1:1 uno a uno

: muchos a muchos

Segunda edición:

Se tendrá en cuenta todo lo antes puesto, pero con código adicional por poner

Ejemplo:

```
package src;

public class ClasePersona {

    private String nombre;
    private int edad;

    public ClasePersona (String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
}
```

al igual que en los diagramas, se pueden clasificar los datos y objetos

```

public String setNombre() {
    return nombre;
}

public int setEdad() {
    return edad;
}

public String getNombre() {
    return nombre;
}

public int getEdad() {
    return edad;
}

public void consultarNombre() {
    System.out.println("hola, mi nombre es " + nombre);
}

public void consultarEdad() {
    System.out.println("hola, mi edad es " + edad);
}

```

Forma de aplicarles herencia/polimorfismo:

```

package src;

public class ClaseDocente extends ClasePersona {

    private String materia;
    private int salario;

    public ClaseDocente(String materia, int salario, String nombre, int edad) {
        super (nombre, edad);
        this.materia = materia;
        this.salario = salario;
    }
}

```